

Physical OOP: Spatial Program Physical Objects Interactions

Xiaoyan Wei*
The University of Adelaide

Zijian Yue†
The University of Adelaide

Hsiang-Ting Chen‡
The University of Adelaide

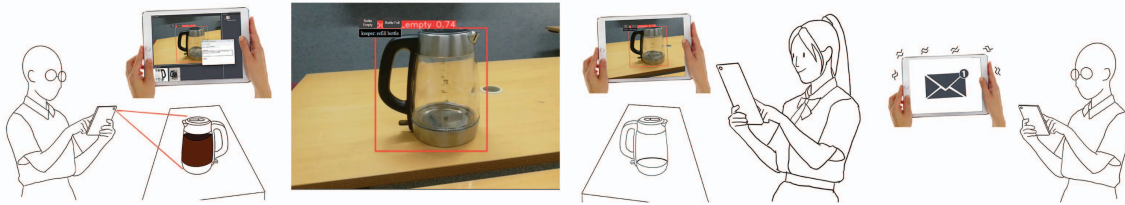


Figure 1: PhyOOP is a new coding environment for spatial computing applications. (1) PhyOOP uses a rich text editor to provide users with the ability to freely write code using images of objects. User 1 uses PhyOOP to write code to monitor the status of the kettle. (2) The code is virtually attached to the kettle. (3) When user 2's device observe the kettle later in the day, the system detect the state changes of the kettle. The user 2 can also read the user 1's code and write her own code. (4) User 1 receiving an email from the PhyOOP system to re-fill the kettle.

ABSTRACT

Spatial computing could change our interaction with the physical world, yet development tools still largely apply to outdated methods, representing 3D objects with 2D text in code editors. This discrepancy makes the programming of spatial applications more complicated. PhyOOP, our Physical Object-Oriented Programming Environment, trying to solve this issue by capturing object states through camera feeds for direct code *insertion* and *attaching* executable snippets to objects for camera-triggered actions. The user study, involving 16 participants, suggests that PhyOOP is easy-to-use and has potential applications in various domains.

Index Terms: Human-centered computing—Human computer interaction (HCI); User interface programming; Augmented reality; Spatial Computing

1 INTRODUCTION

Mixed reality (MR) blends the physical and virtual worlds, enabling real-time interaction between them. With the development of mixed reality, the need to connect virtuality and reality continues to strengthen. People have the ability to communicate precise instructions clearly and conveniently at any time in everyday environments. However, converting such instructions into specific computer programs is a complex challenge in MR environments.

Recent advancements in authoring tools within MR are reshaping the interaction between the physical and virtual worlds. Recent work has identified that novel interaction methods can enhance both interest and efficiency in programming tasks [1, 3]. Zhu et al. [6] have developed a system integrating the Internet of Things (IoT) with virtual reality, employing a block-based programming interface to augment immersive coding education. Concurrently, Monteiro et al. [2] introduce an AR system that empowers users to create functional AR prototypes using an intuitive interface.

Despite significant progress in these authoring tools, numerous challenges persist. In particular, the inadequacy of traditional 2D

programming environments for the demands of 3D spatial computing. These environments typically lack the necessary tools for direct processing and manipulation of 3D objects and environments. Furthermore, existing tools do not adequately support the multi-user experience, which is essential for collaborative spatial computing.

We introduce Physical Object-Oriented Programming (PhyOOP), a spatial programming environment that combines physical object interaction with MR. Designed to integrate human interaction, environmental elements and AI automation. PhyOOP pre-trained object models help users recognize objects through cameras and then convert physical entities into image-based code input, simplifying application design in automation scenarios.

This paper presents two key contributions. Firstly, it introduces a novel concept called PhyOOP, which allows for direct interactions between code and physical objects. Secondly, it details an interactive AR programming system, which enables users to seamlessly integrate the states of physical objects into their code using a drag-and-drop interface for constructing logical statements. The paper also reports a pilot study of PhyOOP system with 16 users.

2 SYSTEM AND IMPLEMENTATION

System In the realm of mixed reality, PhyOOP stands out as a spatial programming environment specifically tailored for the interfacing of physical objects. PhyOOP has a streamlined user interface where the primary window continually reflects the current AR view. It presents the status of a selected object and any associated programs. The status panel, offers a gallery of selectable object states. Adjacent to the status panel is the rich text editor, designed to write programs and encourage the use of object-state imagery.

When an object is selected by the user, all object status images are displayed in the status panel, providing users with convenient choices. To redefine the traditional text editor, we changed the standard plain text input mechanism into a rich text editor that allows users to incorporate images, essentially changing the way users interact with programming. This rich text editor serves as a coding area where users can represent object states through photos and use these images as variables during coding.

When the user completes the program writing and submits it, the system classifies the codes according to the selected object status images and automatically aligns them to the floating status buttons on the objects in the video for the user's convenience.

In the domain of storage and execution solutions, we have proposed a new concept where code manifests as an entity attached

*e-mail: xiaoyan.wei@adelaide.edu.au

†e-mail: zijian.yue@student.adelaide.edu.au

‡e-mail: tim.chen@adelaide.edu.au

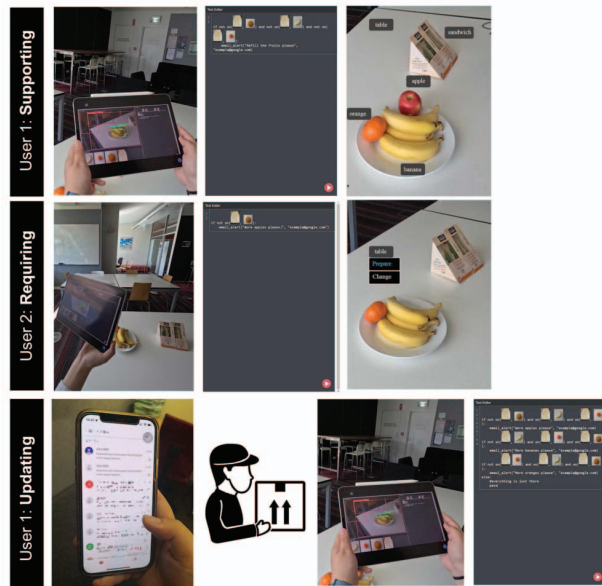


Figure 2: A scenario where two staff and employees who don't know each other using PhyOOP to write different codes on fruits at different times to communicate.

directly to objects, similar to digital post-it notes but much more functionality. These 'attached codes' are in fact executable programs triggered under specified conditions, offering a dynamic interaction interface that thrives on real-time responses.

Different from conventional methods, the programs are represented in the form of state buttons that reduce the space, maintaining a more clear user interface (UI) that promotes ease of selection and interaction. Users retain the autonomy to assign names and append comments to the codes, fostering a collaborative environment where insights can be shared, and feedback can be received.

PhyOOP supports multiple users and objects, facilitates an understanding of object interactions within the system. Its capabilities are showcased through a scenario where staff and employees, unfamiliar with each other, use PhyOOP to inscribe distinct codes on fruits at various times as a form of communication (refer to Figure 2). This example highlights PhyOOP's efficiency in handling programming tasks for specific items, whether by an individual or multiple users.

Implementation Our system is a browser-based application structured into two main components. Firstly, the front-end, which is developed using JavaScript and HTML, serves as the visual interface of the application. It enables users to detect objects, observe their states, and browse or write code associated with a detected object. Secondly, the backend is constructed using Python and operates as a Flask-based web server. This server facilitates image capture and streams detected images to the frontend display, as well as receives the code sent from the frontend. Additionally, it handles the storage of object states and related statistics and is capable of executing the received code from the front-end.

PhyOOP employs a custom-trained YOLOv8 model integrated into the backend of the application to facilitate real-time object and object-state tracking. PhyOOP's frontend features an HTML-based rich text editor with a groundbreaking feature: it allows users to incorporate images directly into the Python code they are writing without disturbing the structure or indentation of the code. Remarkably, these images are treated as Python variables. The code is interpreted by a Python virtual machine, where distinct threads execute different segments of the code, all while avoiding collision.

3 DISCUSSION

Through our user study, 15 of 16 participants felt that using PhyOOP provided them with a new model for writing code. All six programming beginners believed that this spatial interactive programming environment increased their interest in programming and simplified the programming process. Three senior programmers also believe that the programming method of using pictures as variables and writing executable code to attach to objects can help them solve the problem of using complex codes to represent some objects during the programming process. All 16 participants thought this was an interesting system, and they hope that in the future they can see interesting and executable programs left by others on all objects in their daily lives. This not only facilitates their lives, but also brings them curiosity to explore unknown things. This is also consistent with previous research: spatial authoring can lower the barriers to creating functional AR prototypes while also providing a flexible and versatile design experience [2].

4 CONCLUSION

In conclusion, PhyOOP represents a new approach to spatial computing, breaking the boundaries between how we perceive and utilize programming in the virtual and physical fields. By fusing augmented reality and object-oriented programming, we facilitate a shift from traditional programming to enable more immersive and spatially aware interactions. Our research shows that most programmer participants believe that compared to traditional two-dimensional programming environments, PhyOOP connects programming to the real world very well. When we evaluated through the user study, indicate a positive user reception. The feedback underscores the potential and importance of integrating spatial computing concepts into future programming environments. As we progress, we will enhance cross-platform operability through Unity, considering the interface for different levels of immersion [4] and different modalities [5], and provide a richer visual and auditory feedback system. We hope to create richer, immersive, and inclusive user experiences and envision what's next for the PhyOOP project.

REFERENCES

- [1] X. Han, Y. Chen, Q. Feng, and H. Luo. Augmented reality in professional training: A review of the literature from 2001 to 2020. *Applied Sciences*, 12(33):1024, Jan 2022. doi: 10.3390/app12031024
- [2] K. Monteiro, R. Vatsal, N. Chulpongsatorn, A. Parnami, and R. Suzuki. Teachable reality: Prototyping tangible augmented reality with everyday objects by leveraging interactive machine teaching. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, pp. 1–15, 2023.
- [3] J. Pirker, A. Dengel, M. Holly, and S. Safikhani. Virtual reality in computer science education: A systematic review. In *Proceedings of the 26th ACM Symposium on Virtual Reality Software and Technology, VRST '20*. Association for Computing Machinery, New York, NY, USA, 2020. doi: 10.1145/3385956.3418947
- [4] S. Shen, H.-T. Chen, W. Raffé, and T. W. Leong. Effects of level of immersion on virtual training transfer of bimanual assembly tasks. *Frontiers in Virtual Reality*, 2:597487, 2021.
- [5] R. Wu and H.-T. Chen. The effect of visual and auditory modality mismatching between distraction and warning on pedestrian street crossing behavior. In *2023 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pp. 1045–1054. IEEE Computer Society, Los Alamitos, CA, USA, oct 2023. doi: 10.1109/ISMAR59233.2023.00121
- [6] Z. Zhu, Z. Liu, Y. Zhang, L. Zhu, J. Huang, A. M. Villanueva, X. Qian, K. Peppler, and K. Ramani. LearnIoTVR: An End-to-End virtual reality environment providing authentic learning experiences for internet of things. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, number Article 447 in CHI '23, pp. 1–17. Association for Computing Machinery, New York, NY, USA, Apr. 2023.